

IMOSS explore oracles: the two winning configurations

Exact specification of IMOSS-mutate-KLxTPE and IMOSS-TabPFN, as measured

4 August 2026

Scope

This document specifies, precisely and reproducibly, the two best explore oracles found for IMOSS across five benchmark families. Every parameter below was read back from a live constructed optimizer, not transcribed from memory. Anything not listed is the constructor default.

Two oracles are specified:

- **IMOSS-mutate-KLxTPE** — surrogate-free. The most *reliable* method: first or tied-first on every family measured.
- **IMOSS-TabPFN** (tuned) — surrogate-based. First on four of the six cells measured, second on the two others. A **single** configuration: the earlier per-benchmark quantile rule is superseded by $q = 0.975$ (§“Choosing the quantile”).

Shared framework

Both are explore oracles for IMABO (`imabo/optimizer.py`); they change only how a *new* arm is proposed. Everything else is common:

- **Switching rule** — at round t with $|A_t|$ arms opened so far, the optimizer explores iff $|A_t| < t^\beta$, otherwise it exploits. `switch_strategy="beta"`.
- **Exploitation** — pull the arm maximising the MOSS-anytime index (`moss_anytime, alpha=0.1`), unchanged in both.
- $\beta = 0.5$. Measured across every method: $\beta = 0.8$ is much worse.
- **Warm-up** — the oracle returns uniform random configurations until at least 10 arms have a reward.

A “good/bad” split used by both TPE components ranks rewarded arms by their MOSS-anytime score and takes the top $\lceil 0.3K \rceil$ as good (`default_gamma`), the rest as bad, with at least one arm on each side.

Configuration 1 — IMOSS-mutate-KLxTPE

```
from imabo import IMABOCoordUCB
```

```
IMABOCoordUCB(  
    search_space=...,  
    seed=...,  
    beta=0.5,  
    parent_rule="best",  
    coord_rule="ucb",  
    value_rule="tpe",  
    credit_rule="arm_mean",
```

```

    bandit_bonus="kl",
)

```

Resolved state (including defaults that matter):

parameter	value
parent_rule	"best"
coord_rule	"ucb"
value_rule	"tpe"
credit_rule	"arm_mean"
bandit_bonus	"kl"
bandit_rule	"ucb"
bandit_discount	1.0 (no forgetting)
coord_bandit_scope	"global" (one bandit for the run)
coord_alpha	1.0 (unused under KL)
require_new_arm	False
mutation_size	1
global_tpe_arm	False
tpe_value_pick	"ei_argmax"
min_arms_for_mutation	10
n_ei_candidates	24
prior_weight	1.0

What one explore step does

1. **Guard.** If fewer than $\max(2, 10)$ rewarded arms exist, return a uniform random configuration and stop.
2. **Refresh credits.** Because `credit_rule="arm_mean"`, every previously registered decision is re-scored from the current arm means *before* any bandit index is read. This is lazy on purpose: nothing reads an index between oracle calls, so re-scoring per reward would be invisible work. Verified to give selections identical to a per-reward implementation.
3. **Parent.** The arm with the highest empirical mean — an incumbent hill-climb. Note this is *not* the arm MOSS is pulling; the two argmaxes agreed only 58% of the time on the RF grid.
4. **Coordinate.** A KL-UCB bandit over the d coordinates. Any coordinate never credited has index $+\infty$, so the first d decisions are a forced round-robin; afterwards the index of coordinate i is the largest $q \in [\hat{\mu}_i, 1]$ with $n_i \text{KL}(\hat{\mu}_i, q) \leq \log t$, where t counts this bandit's own selections.
5. **Value.** A univariate TPE on that one coordinate: fit 1-D Parzen densities ℓ (good arms) and g (bad arms), draw 24 candidate values from ℓ , and take the one maximising ℓ/g . Values equal to the parent's current value are skipped.
6. **Serve.** The mutant is the parent with that one coordinate replaced. It is returned **even if that configuration is already an open arm** (see §“Already-open arms”).
7. **Register.** Record a decision (coordinate, child key, parent key). Its single vote carries the child's running mean, revised in place at each later refresh via `UCB1.revise` so the vote *count* stays at one while its *value* sharpens.

Why each choice, in one line each

- `parent_rule="best"` — the largest single effect in the family: -26% vs a softmax parent for this oracle.
- `coord_rule="ucb"` — having a coordinate bandit at all is worth -117.9 ± 28.1 ($t = -4.19$) against choosing uniformly. This is the dominant term.
- `bandit_bonus="kl"` / `credit_rule="arm_mean"` — both point the right way but neither is individually significant (-19.4 ± 18.7 and -13.3 ± 26.9); their interaction is -3.1 ± 29.0 , i.e. exactly additive. Chosen on direction, not on evidence. `Hoeffding + arm_mean` is a defensible lower-variance alternative (± 95 vs ± 137).

- Everything else is a default that was tested and left alone: discounting (+43.8, +52.9), EXP3 (+59.5), EXP3.S (+117.0), per-parent bandits (+14.2), forced novelty (−0.8, but 5× fewer oracle calls), multi-coordinate mutation (+55.7, +61.8).

Configuration 2 — IMOSS-TabPFN (tuned)

```
from imabo import IMABOTabPFN
```

```
IMABOTabPFN(
    search_space=...,
    seed=...,
    beta=0.5,
    candidate_source="mutation",
    parent_rule="best",
    candidate_uniform_frac=0.1,
    mutation_scale=0.1,
    refit_every=1,
    quantile=0.975,
    tabpfn_model=...,
)
```

Resolved state:

parameter	value
candidate_source	"mutation"
parent_rule	"best"
n_candidates	100
candidate_uniform_frac	0.1 (→ 10 uniform, 90 mutants)
mutation_size	1
mutation_scale	0.1
refit_every	1
acquisition	"quantile"
quantile	0.975
filter_open_candidates	True
n_estimators	4
max_num_rows	200
fit_granularity	"arm"
min_arms_for_fit	10
candidate_tpe_frac	0.0
candidate_topup	"uniform" (unused: mix source only)

What one explore step does

1. **Guard.** Fewer than 10 rewarded arms → uniform random configuration.
2. **Shortlist.** If candidates are queued from a previous fit, pop the next one. At `refit_every=1` the queue is always empty, so this never fires.
3. **Pool (100 candidates).** 10 uniform draws over the whole space, plus 90 mutants of the incumbent (highest empirical mean). Each mutant changes exactly one uniformly-chosen coordinate:
 - **numeric axis** — a Gaussian step of `mutation_scale` × the axis width, taken in log space for log-scaled parameters, then clamped to the domain *in the original space*;
 - **categorical axis** — uniform over the other levels (`mutation_scale` has no effect; the call is bit-identical to the unscaled path).
4. **Hygiene.** Deduplicate by arm key, then drop every candidate already in memory. If nothing survives, redraw uniformly.

5. **Fit.** One TabPFN-3 regressor on one row per rewarded arm (configuration $\rightarrow$ mean reward), 4 estimators, at most 200 rows.
6. **Score and serve.** Rank candidates by the 0.975 quantile of the predictive distribution; return the argmax.

The three settings that matter

Ordered by measured effect. All three differ from the shipped defaults.

change	effect	where measured
<code>refit_every</code> 10 $\rightarrow$ 1	-98.9 ± 17.4 , $t = -5.69$	RF grid
<code>quantile</code> 0.99 $\rightarrow$ 0.975	-42.8 ± 7.5 / -508.5 ± 208.7	svm / toys
<code>mutation_scale</code> none $\rightarrow$ 0.1	-26	svm

refit_every. The default 10 caches the top 10 scored candidates and serves them over the next nine explore steps without refitting. Measured at 5000 rounds that means TabPFN is fit **7 times per run** and 89% of proposals come off a shortlist built before the arms preceding them existed. Setting it to 1 removes the staleness entirely. It is also the parameter that made every earlier pool-composition comparison unreadable: five structurally different pools span 73 regret points at `refit_every`=10 and only 16.6 at `refit_every`=1.

quantile. The acquisition score is a quantile of the predictive distribution, so it rewards predictive variance. Whether that is safe depends entirely on the geometry of the pool:

- On a **discrete grid**, all 100 candidates sit inside one 25-configuration neighbourhood, variance is near-uniform, and the quantile behaves like a ranking by mean.
- On a **continuous box** spanning five decades, variance is wildly heterogeneous and a high quantile systematically picks the *least familiar* candidate — exploration piled on top of a switching rule that already explores.

See §“Choosing the quantile” for the full sweep; 0.975 serves both regimes.

mutation_scale. Without it, `mutate_value` resamples a continuous axis log-uniformly over its whole domain. Measured on the 2-D HPO boxes, a “mutant” then lands a mean 0.25 of the axis’ full log-range from the parent — 1.25 decades on a 5-decade axis — with only 19% within 10% of it. That is a global search along two axis-aligned lines, not a neighbourhood. At `scale`=0.1 the mean distance falls to 0.079 and 69% of mutants land within 10%.

Choosing the quantile

The quantile is the only parameter whose best value differs by benchmark, so it was swept on all six cells at matched `mutation_scale` and `refit_every`=1. It is equivalent to a UCB weight $\kappa = \Phi^{-1}(q)$: 0.841 $\rightarrow$ 1.00, 0.9 $\rightarrow$ 1.28, 0.975 $\rightarrow$ 1.96, 0.99 $\rightarrow$ 2.33.

cell	$q=0.99$	$q=0.975$	$q=0.9$	$q=0.841$	best
RF grid	438.0	435.0	468.6	477.8	0.975
toys	8100.6	7592.1	7978.7	8756.9	0.975
family_d2	308.7	307.1	563.1	622.5	0.975
prod_d2	229.0	195.7	249.6	290.9	0.975
svm T $\leq$ 3k	174.0	131.2	–	103.2	0.841
lr T $\leq$ 3k	–	178.6	–	157.0	0.841
geom. mean vs per-cell best	1.163	1.063	–	1.250	

$q = 0.975$ is the fixed value. It is first on four of six cells and within 6.3% of the best everywhere, against 16.3% for 0.99 and 25.0% for 0.841. It weakly dominates 0.99 – equal or better on all five cells where both were run.

Two structural facts fall out of the sweep.

A cliff between $\kappa = 1.96$ and $\kappa = 1.28$. On **family_d2**, 0.975 matches 0.99 (-1.6 ± 51.9) while 0.9 loses by $+254.4 \pm 91.9$ ($t = +2.77$). Those landscapes need a large optimism bonus or the acquisition never picks a global candidate at all (§“Where the tuning fails”). 0.975 is the least optimism that still clears the cliff.

Only log-scaled multi-decade axes prefer less. **lr** (2 axes, 5 decades) and **svm** (2 axes, 6 decades) are the only cells favouring 0.841; the toys (linear $[-5.12, 5.12]$), **family_d2** (linear $[0, 1]$) and the RF grid (categorical) all prefer $\kappa \geq 1.96$. It is the axis *scaling*, not the dimension: **family_d2** is also 2-D and wants the high value. On those two cells 0.975 still recovers 60% of what 0.99 gave away.

Results

Cumulative regret, lower is better. RF grid: sum over 3 OpenML tasks, 30 paired seeds. **lr**/**svm**: sum over 4 budgets, 20 seeds. Toys: sum over 3 functions $\times$ 4 budgets, 20 seeds.

All TabPFN numbers at $q = 0.975$, the single fixed configuration.

family	KLxTPE	TabPFN	paired difference	t
RF tabular grid (discrete, 4-D)	516.6	435.0	-81.6 ± 27.6 , 18/30	-2.95
toys (continuous, 4-D)	8174.3	7592.1	-582.1 ± 314.7 , 14/20	-1.85
barrier family_d2	499.1	307.1	-192.0 ± 79.3 , 23/30	-2.42
barrier prod_d2	198.7	195.7	-2.9 ± 30.5 , 14/30	-0.10
svm T $\leq 3k$ (2-D, 6 dec.)	105.8	131.2	$+25.4 \pm 5.9$, 3/20	+4.28
lr T $\leq 3k$ (2-D, 5 dec.)	166.2	178.6	$+12.4 \pm 5.6$, 6/20	+2.21

TabPFN wins four cells (two significantly), ties **prod_d2**, and loses only the two log-scaled continuous boxes. KLxTPE remains the lower-variance choice and needs no surrogate at all; TabPFN is the stronger one wherever a pool of candidates can usefully be ranked.

For reference, the best non-IMOSS baselines: **Stroquool** wins **lr** (650.9) but is $4\times$ worse on **svm**; **Hier-MAB** and the tree methods are behind on every family.

Where the tuning fails

At $q = 0.841$ the oracle failed badly on the coordination-barrier landscapes: 622.5 on **family_d2**, worse than every alternative including vanilla TabPFN (uniform pool, **refit_every**=10, $q=0.99$) at 368.9. The cause is worth recording because it is not what it looks like.

candidate_uniform_frac=0.1 puts 10 uniform draws in every pool, so global candidates are always *available*. They are simply never *chosen*. Instrumented on **family_d2** over 3 seeds $\times$ 5000 rounds, all at **scale**=0.1, **refit_every**=1:

acquisition	explore steps	chose a uniform candidate
$q = 0.841$	183	0 (0.0%)

acquisition	explore steps	chose a uniform candidate
$q = 0.99$	183	22 (12.0%)

A far-away candidate has a *low predicted mean*: TabPFN’s training table holds only arms from the near side of the barrier, and nothing in it implies that a coordinated move pays off. The only thing that can carry such a candidate to the argmax is the optimism bonus from its high predictive variance. Lowering the quantile deletes that bonus and the uniform share becomes decoration – the oracle is 100% local in practice, on a landscape whose entire design punishes locality. Vanilla TabPFN wins there not because a uniform pool is better in principle, but because with 100% uniform candidates the acquisition has nothing local to prefer.

candidate_uniform_frac is an exploration offer, not a guarantee.

This does not call for a uniform pool. An earlier draft of this document recommended one for barrier landscapes on exactly that vanilla-beats-tuned evidence. That was wrong: with the quantile restored, the *mutation* pool at `refit_every=1` beats vanilla on both landscapes (-60.3 ± 44.7 on `family_d2`, -14.0 ± 15.4 on `prod_d2`). Nor is `mutation_scale` implicated – at fixed $q=0.99$ the local step is neutral there (302.8 vs 308.7, 219.3 vs 229.0). The quantile alone explains the failure, and $q = 0.975$ resolves it completely: 307.1 on `family_d2`, the best of any method measured.

Already-open arms: the two oracles differ

This is the largest behavioural difference between them and is worth stating explicitly.

KLxTPE re-proposes freely. With `require_new_arm=False` the mutant is served whether or not it already exists. Measured over 5 seeds $\times$ 5000 rounds on the RF grid:

task	oracle calls	proposal already open	arms opened
segment	1857	97.4%	59.0
credit-g	1667	96.9%	61.8
numera1	1114	95.0%	66.0

This is self-sustaining: a repeat does not grow $|A_t|$, so $|A_t| < t^\beta$ stays true and the oracle fires again next round — which is why it receives ~1100–1900 calls rather than the ~112 the switching rule nominally allows. The round is not wasted (it pulls a mutation of the incumbent), but each repeat registers a *fresh* decision, so one configuration casts 17–32 votes on the coordinate bandit. Those duplicate votes are what let its intervals shrink and the bandit commit. Forcing novelty instead (`require_new_arm=True`) gives the same regret (515.8 vs 516.6) with $\sim 5\times$ fewer oracle calls and lower variance — a compute win, not a quality win.

TabPFN never re-proposes. `filter_open_candidates=True` drops already-open candidates before scoring, so every explore step opens a new arm: 61 explore steps $\rightarrow$ 71 arms, hitting the t^β target exactly, versus 43–71 (mean 66.5) without it. The filter is not rescuing an exhausted pool — the pool almost always contains novel candidates — it *overrides the acquisition*, which on a finite grid reliably ranks an already-open neighbour above any unopened candidate. Its cost is regime-dependent: +72.5 at `refit_every=10`, but that was measured against a nine-tenths-stale shortlist, and at `refit_every=1` the filtered configuration is the best TabPFN result obtained. On continuous spaces it is a no-op, since mutants never collide.

Reproduction

On this branch both oracles are wired into every experiment under the names IMOSS-mutate-KLxTPE and IMOSS-TabPFN-tuned:

- Discrete grid: `python experiments/rf_arm_distribution_experiment.py --algorithm IMOSS-TabPFN-tuned --n-runs 30 --n-iter 5000 --n-shadow 0`

- Continuous HPO: `python experiments/hpo_experiment.py --benchmarks lr svm`
- Toys: `python experiments/toy_experiment.py`
- Barrier: `python experiments/coordination_barrier_experiment.py`

Coverage gap: the `lr/svm` cells are measured at $T \leq 3000$ only; the 5000 and 10000 budgets have not been run at $q = 0.975$.

Every number in this document was produced on the `oracles-archive` branch, which holds the full ablation. The two configurations were then ported here and verified to reproduce those runs **bit-for-bit** (`rf31` seeds 42 and 43, both oracles).

`--n-shadow 0` disables the oracle-quality probe, which is otherwise ~75–80% of all TabPFN calls in a run and leaves the regret trajectory bit-identical (the probe runs on a deep copy).